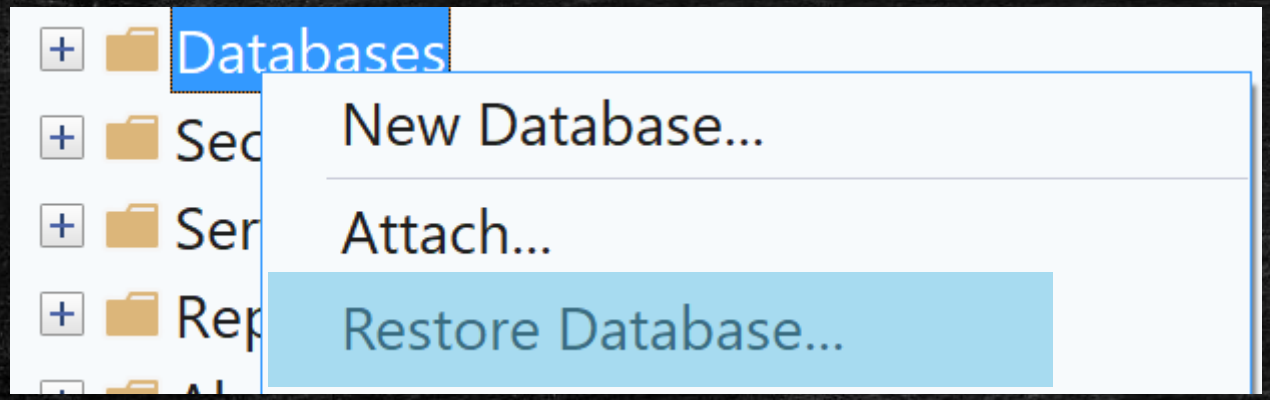
Adham Ashraf Saeed

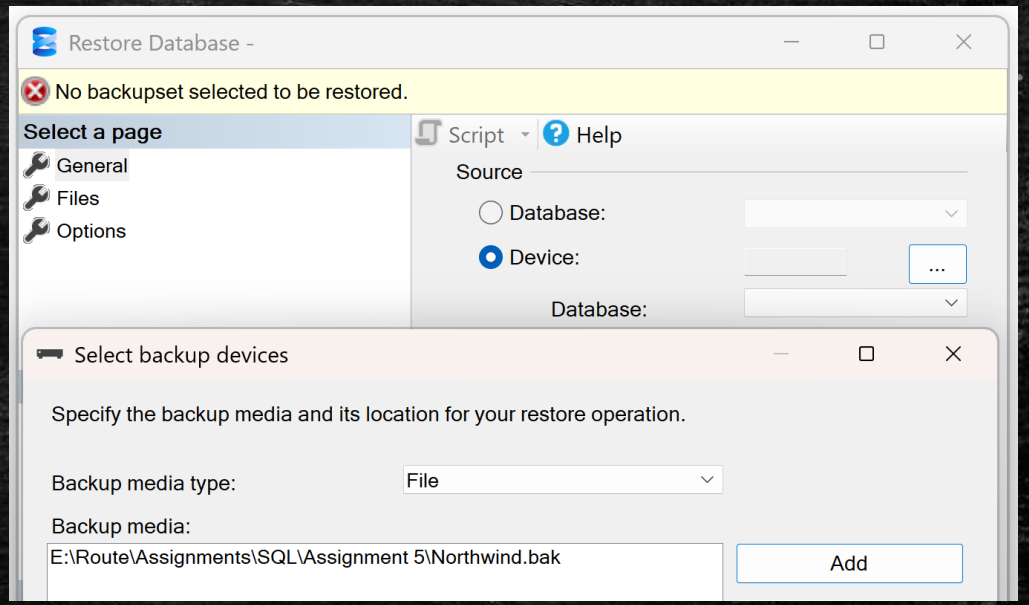
Assignment 5 (SQL)

Load the provided dataset into your database

- Open the provided links and download the dataset files

Northwind






```
USE NORTHWND;
```

Questions

1. Write a query to rank each customer based on the total value of their orders, with 1 being the highest order value.

```sql
WITH    OrderValue
AS      (SELECT    C.CustomerID,
                   C.ContactName,
                   SUM(D.UnitPrice * D.Quantity * (1 - Discount)) AS TotalSales
         FROM      Customers AS C
                   INNER JOIN
                   Orders AS O
                   ON O.CustomerID = C.CustomerID
                   INNER JOIN
                   OrderDetails AS D
                   ON D.OrderID = O.OrderID
         GROUP BY C.CustomerID, C.ContactName)
SELECT CustomerID,
       ContactName,
       TotalSales,
       RANK() OVER (ORDER BY TotalSales DESC) AS Ranking
FROM   OrderValue;
```

2. Write a query to find the second highest unit price of products in each category.

```sql
WITH   HighestUP
AS     (SELECT CategoryID,
               ProductID,
               ProductName,
               UnitPrice,
               DENSE_RANK() OVER (PARTITION BY CategoryID ORDER BY UnitPrice DESC) AS Ranking
        FROM   Products AS P)
SELECT CategoryID,
       ProductID,
       ProductName,
       UnitPrice
FROM   HighestUP
WHERE  Ranking = 2;
```

Questions

3. Write a query to calculate the total sales amount each employee made and include a running total (cumulative sum) of these sales

```
WITH    EmployeeTS
AS      (SELECT    E.EmployeeID,
                   CONCAT(E.FirstName, ' ', E.LastName) AS FullName,
                   SUM(D.UnitPrice * D.Quantity * (1 - Discount)) AS TotalSales
         FROM      Employees AS E
                   INNER JOIN
                   Orders AS O
                   ON O.EmployeeID = E.EmployeeID
                   INNER JOIN
                   OrderDetails AS D
                   ON D.OrderID = O.OrderID
         GROUP BY E.EmployeeID, CONCAT(E.FirstName, ' ', E.LastName))
SELECT EmployeeID,
       FullName,
       TotalSales,
       SUM(TotalSales) OVER (    ORDER BY EmployeeID
                                 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                            ) AS CumulativeTS
FROM   EmployeeTS;
```

4. Write a query to find out the employees whose average order value is higher than the average order value of all employees.

```
WITH      EmployeeOrders
AS        (SELECT    E.EmployeeID,
                     CONCAT(E.FirstName, ' ', E.LastName) AS FullName,
                     O.OrderID,
                     SUM(D.UnitPrice * D.Quantity * (1 - D.Discount)) AS OrderValue
           FROM      Employees AS E
                     INNER JOIN
                     Orders AS O
                     ON O.EmployeeID = E.EmployeeID
                     INNER JOIN
                     OrderDetails AS D
                     ON D.OrderID = O.OrderID
           GROUP BY E.EmployeeID, CONCAT(E.FirstName, ' ', E.LastName), O.OrderID)
SELECT    EmployeeID,
          FullName,
          AVG(OrderValue) AS AvgOrderValue
FROM      EmployeeOrders
GROUP BY EmployeeID, FullName
HAVING    AVG(OrderValue) > (SELECT AVG(OrderValue)
                             FROM   EmployeeOrders);
```

Questions

5. Write a query to find the employee who processed the most orders each year.

```sql
WITH    EmployeeOrders
AS      (SELECT    E.EmployeeID,
                   CONCAT(E.FirstName, ' ', E.LastName) AS FullName,
                   YEAR(O.OrderDate) AS OrderYear,
                   COUNT(DISTINCT O.OrderID) AS TotalOrders
         FROM      Employees AS E
                   INNER JOIN
                   Orders AS O
                   ON O.EmployeeID = E.EmployeeID
         GROUP BY E.EmployeeID, CONCAT(E.FirstName, ' ', E.LastName), YEAR(O.OrderDate)),
        YearRanking
AS      (SELECT EmployeeID,
                FullName,
                OrderYear,
                TotalOrders,
                RANK() OVER (PARTITION BY OrderYear ORDER BY TotalOrders DESC) AS Ranking
         FROM   EmployeeOrders)
SELECT *
FROM   YearRanking
WHERE  Ranking = 1;
```

6. Write a query to find the top 3 most expensive products in each category.

```sql
WITH    EXP_Products
AS      (SELECT CategoryID,
                ProductID,
                ProductName,
                UnitPrice,
                DENSE_RANK() OVER (PARTITION BY CategoryID ORDER BY UnitPrice DESC) AS Ranking
         FROM   Products AS P)
SELECT CategoryID,
       ProductID,
       ProductName,
       UnitPrice
FROM   EXP_Products
WHERE  Ranking <= 3;
```

Questions

7. Rank products based on their total sales amount inside each category, with rank 1 being the highest-selling product per category.

```sql
WITH    ProductSales
AS      (SELECT    P.CategoryID,
                   P.ProductID,
                   P.ProductName,
                   SUM(O.UnitPrice * O.Quantity * (1 - Discount)) AS TotalSales
         FROM      Products AS P
                   INNER JOIN
                   OrderDetails AS O
                   ON O.ProductID = P.ProductID
         GROUP BY P.CategoryID, P.ProductID, P.ProductName)
SELECT CategoryID,
       ProductID,
       ProductName,
       TotalSales,
       RANK() OVER (PARTITION BY CategoryID ORDER BY TotalSales DESC) AS Ranking
FROM   ProductSales;
```

8. Find the highest order value handled by each employee and return the corresponding order.

```sql
WITH    EmployeeOrders
AS      (SELECT    E.EmployeeID,
                   CONCAT(E.FirstName, ' ', E.LastName) AS FullName,
                   O.OrderID,
                   SUM(D.UnitPrice * D.Quantity * (1 - Discount)) AS OrderValue
         FROM      Employees AS E
                   INNER JOIN
                   Orders AS O
                   ON O.EmployeeID = E.EmployeeID
                   INNER JOIN
                   OrderDetails AS D
                   ON D.OrderID = O.OrderID
         GROUP BY E.EmployeeID, CONCAT(E.FirstName, ' ', E.LastName), O.OrderID),
       EmployeeRanking
AS      (SELECT EmployeeID,
                FullName,
                OrderValue,
                OrderID,
                ROW_NUMBER() OVER (PARTITION BY EmployeeID ORDER BY OrderValue DESC) AS Ranking
         FROM   EmployeeOrders)
SELECT EmployeeID,
       FullName,
       OrderValue,
       OrderID
FROM   EmployeeRanking
WHERE  Ranking = 1;
```

Questions

9. Calculate total sales per category and show each category's percentage of total sales

```sql
WITH    SalesCategory
AS      (SELECT    C.CategoryID,
                   C.CategoryName,
                   SUM(O.Quantity * O.UnitPrice * (1 - Discount)) AS TotalSales
         FROM      Categories AS C
                   INNER JOIN
                   Products AS P
                   ON C.CategoryID = P.CategoryID
                   INNER JOIN
                   OrderDetails AS O
                   ON O.ProductID = P.ProductID
         GROUP BY C.CategoryID, C.CategoryName)
SELECT CategoryID,
       CategoryName,
       TotalSales,
       TotalSales * 100.0 / SUM(TotalSales) OVER () AS PercentageOfSales
FROM   SalesCategory;
```

10. Find customers whose total order value is above the average customer order value

```sql
WITH    CustomerOrders
AS      (SELECT    C.CustomerID,
                   C.ContactName,
                   SUM(D.Quantity * D.UnitPrice * (1 - Discount)) AS TotalSales
         FROM      Customers AS C
                   INNER JOIN
                   Orders AS O
                   ON O.CustomerID = C.CustomerID
                   INNER JOIN
                   OrderDetails AS D
                   ON D.OrderID = O.OrderID
         GROUP BY C.CustomerID, C.ContactName)
SELECT CustomerID,
       ContactName,
       TotalSales
FROM   CustomerOrders
WHERE  TotalSales > (SELECT AVG(TotalSales)
                     FROM   CustomerOrders);
```

Questions

11. Find products that have never been ordered

```sql
SELECT P.ProductID,
       ProductName
FROM   Products AS P
       LEFT JOIN
       OrderDetails AS O
       ON O.ProductID = P.ProductID
WHERE  O.ProductID IS NULL;
```

12. For each employee, calculate total sales per year

```sql
WITH     EmployeeSales
AS       (SELECT   E.EmployeeID,
                   CONCAT(E.FirstName, ' ', E.LastName) AS FullName,
                   YEAR(O.OrderDate) AS OrderYear,
                   SUM(D.Quantity * D.UnitPrice * (1 - Discount)) AS SalesPerYear
          FROM     Employees AS E
                   INNER JOIN
                   Orders AS O
                   ON O.EmployeeID = E.EmployeeID
                   INNER JOIN
                   OrderDetails AS D
                   ON D.OrderID = O.OrderID
          GROUP BY E.EmployeeID, CONCAT(E.FirstName, ' ', E.LastName), YEAR(O.OrderDate))
SELECT   EmployeeID,
         FullName,
         OrderYear,
         SalesPerYear
FROM     EmployeeSales
ORDER BY EmployeeID, OrderYear;
```